Design and Optimization of a Dense Atrous Network for Semantic Segmentation

Diana Cristina Andrade Damian
dianacristina.andradedamian@studenti.unipd.it

Diego Alonso Brule Galleguillos
diegoalonso.brulegalleguillos@studenti.unipd.it

Marina Lima Braga
marina.limabraga@studenti.unipd.it

Abstract

Semantic segmentation aims to assign a semantic label to every pixel in an image, essentially labeling different elements present in it, and plays a key role in applications such as autonomous driving, virtual try-on systems, and human–computer interaction. In this project, we study and implement a state-of-the-art semantic segmentation DenseNetASPP architecture based on densely, recursively connected atrous convolutions, which differs from the usual convolutional approach to computer vision, to address the challenge of scale variation in complex scenes. We start by reconstructing a DenseNetASPP model from the literature with the full network pipeline and analyse its main components, with particular attention to the role of multi-scale context aggregation. We then explore other architectures that may be relevant to the problem with the goal of improving segmentation accuracy while preserving computational efficiency. Experimental results showed a potential in the implementation of encoder-decoder system for the refining of pre-trained models' weights, such as DenseNet121, due to its ability to preserve low-level features. These findings can be the basis of further research to explore the effects of encoding systems in semantic segmentation problems. Overall, this work provides both faithful reproduction and a meaningful extension of the original approach, offering insights into the practical limitations of the model and possible further improvement opportunities.

1. Introduction

In recent years, deep learning has significantly advanced the performance of semantic segmentation models, with fully convolutional networks and encoder–decoder architectures becoming the dominant paradigm. However, segmentation in complex scenes remains difficult due to large variations in object scale, appearance, and spatial context. In urban environments, for example, very small objects such as distant pedestrians or traffic signs coexist with large structures like buildings and roads, often within the same image. Successfully handling such diversity requires models capable of capturing both fine local details and broader contextual information.

To address these challenges, modern architectures increasingly rely on multi-scale feature extraction and context aggregation mechanisms. Among these, atrous (or dilated) convolutions and spatial pyramid pooling strategies have proven particularly effective, as they allow the receptive field to be enlarged without sacrificing spatial resolution. The Densely Connected Atrous Spatial Pyramid Pooling (DenseASPP) architecture represents a significant step in this direction by densely connecting atrous convolutional layers, enabling a more continuous and expressive sampling of the scale space.

In this work, we study, reproduce, and optimize a dense atrous-based semantic segmentation architecture inspired by DenseASPP. Our objective is not only to understand its core design principles, but also to investigate how architectural and training modifications can improve segmentation performance while maintaining computational efficiency.

1.1. Problem Definition

Semantic segmentation is a fundamental task in computer vision that aims to assign a semantic class label to each pixel in an image. Unlike image classification or object detection, which operate at the image or object level, semantic segmentation requires a fine-grained understanding of the entire scene, making it particularly challenging from both a modeling and computational perspective. This capability is crucial in a wide range of real-world applications, including autonomous driving, medical image analysis, augmented reality, and human–computer interaction, where accurate pixel-level predictions directly affect system reliability and safety. The core problem addressed in this project is the design and optimization of an architecture capable of effectively capturing multi-scale contextual information while preserving spatial accuracy and computational feasibility.

Standard convolutional neural networks exhibit inherent

limitations in this setting. As receptive fields grow with network depth, high-level layers become increasingly semantic but lose spatial precision due to pooling and downsampling operations. Conversely, early layers preserve spatial detail but lack sufficient semantic abstraction. This creates a fundamental trade-off between localization accuracy and contextual understanding, which is particularly critical in pixel-level prediction tasks.

Multi-scale processing techniques attempt to mitigate this issue by enabling the network to operate over multiple receptive field sizes. However, many existing approaches rely on sparse parallel structures or introduce significant computational overhead. As a result, there is a need for architectures that can densely and efficiently encode scale diversity without compromising resolution or efficiency.

Within this context, this project focuses on evaluating and optimizing a dense atrous-based segmentation network, analyzing how architectural design choices and hyperparameter configurations influence its ability to balance contextual richness, spatial precision, and computational cost.

1.2. Motivation and Relevance

Despite the substantial progress achieved by deep learning in semantic segmentation, understanding how architectural design choices influence model behavior remains an open and practically relevant challenge. While state-of-the-art architectures often achieve impressive benchmark results, their performance is highly sensitive to factors such as dilation rates, feature aggregation strategies, and training configurations, which are not always systematically analyzed.

This project is motivated by the need to move beyond a purely performance-driven application of existing models and toward a deeper understanding of their internal mechanisms and design trade-offs. In particular, DenseASPP [5] provides an interesting case study due to its dense connectivity pattern and continuous multi-scale representation, making it a suitable framework for exploring how scale diversity can be efficiently encoded.

From both a practical and educational perspective, reproducing and optimizing such an architecture allows us to bridge the gap between theoretical architectural concepts and real-world implementation constraints. By systematically investigating the impact of architectural and training choices on segmentation quality, robustness, and efficiency, this work contributes to a more informed and transparent use of deep segmentation models in real-world applications.

1.3. Overview of Contributions and Results

Overall, it was possible to identify which approaches did not improve the main performance measure, meanIoU. For example, using CBAM produced the opposite of the intended effect, as it decreased performance compared to the vanilla version created from the original structure. In contrast, architectures based on an encoder-decoder system demonstrated that small improvements are possible, although not significant enough to justify the parameter increase.

2. Related Work

Semantic segmentation has been extensively studied in computer vision, with significant progress driven by deep convolutional architectures that enable pixel-level prediction. Early approaches based on fully convolutional networks were later extended with mechanisms to capture broader contextual information, addressing the challenges posed by large variations in object scale and scene complexity. In this context, multi-scale feature extraction strategies based on atrous convolutions and pyramid pooling have become central to state-of-the-art segmentation models. This section reviews the most relevant work in this area, with a particular focus on DenseASPP, which serves as the primary architectural reference for this study.

2.1. DenseASPP for Semantic Segmentation in Street Scenes

Recent advances in semantic segmentation have been largely driven by deep convolutional architectures capable of capturing multi-scale contextual information. Fully Convolutional Networks (FCNs) structures laid the foundation for pixel-level prediction, while subsequent works introduced strategies to enlarge the receptive field without sacrificing spatial resolution, such as atrous (dilated) convolutions and pyramid pooling mechanisms.

Among these approaches, the Dense Atrous Spatial Pyramid Pooling (DenseASPP) [5] architecture represents a notable contribution to multi-scale feature aggregation in urban scene segmentation. DenseASPP extends the idea of Atrous Spatial Pyramid Pooling by densely connecting atrous convolutional layers with increasing dilation rates, enabling a more continuous and expressive sampling of the scale space. The original work evaluates this architecture on the Cityscapes dataset and demonstrates strong performance in capturing objects with large scale variations.

Our work is closely related to DenseASPP, as it adopts the same architectural backbone and is built following the design principles introduced in the original paper. In particular, we reproduce the dense atrous-based structure and conduct our experiments on the Cityscapes dataset, ensuring comparability with previously reported results.

2.2. CBAM: Convolutional Block Attention Module

The Convolutional Block Attention Module (CBAM) is a lightweight, general-purpose attention mechanism designed to improve the representation power of CNNs by focusing on significant features and suppressing less useful ones [4].

It operates by sequentially inferring attention maps across two distinct dimensions: channel and spatial [4]. In our implementation, CBAM functions as a feature refiner positioned immediately after the DenseASPP block but before the final classification head.

The logic behind this placement is to treat the module as a selective filter for the high-dimensional feature maps generated by the DenseASPP layers. While DenseASPP effectively captures multi-scale contextual information through dilated convolutions, the resulting feature maps can contain redundant or noisy information. By applying channel attention, our model identifies which of the 512 feature channels are most relevant to the target urban categories, while the subsequent spatial attention module highlights the most informative regions within those channels. This approach allows the model to "clean up" the dense feature representations, ensuring that the final 1×1 convolution operates on a refined set of signals, thereby improving the precision of the semantic segmentation output.

2.3. Rethinking Atrous Convolution for Semantic Image Segmentation

A significant challenge in semantic segmentation is the loss of spatial detail caused by successive pooling operations and strided convolutions. To address this, [1] introduced the concept of Atrous Spatial Pyramid Pooling (ASPP), which uses parallel atrous convolutions with different rates to capture multi-scale context. While this architecture is highly effective at identifying "what" is in an image, the downsampling process, often reducing the resolution by a factor of 8 or 16, results in blurred object boundaries. In our work, we implement a decoder structure that acts as a bridge between high-level semantic features and low-level spatial details, inspired by the refinements suggested in subsequent iterations of the DeepLab framework [1].

Our approach differs from a standard bottleneck-to-output flow by incorporating a skip connection from the early layers of the DenseNet121 backbone. Specifically, we extract high-resolution feature maps from the beginning of the network and concatenate them with the upsampled output of our CBAM-refined DenseASPP block. The logic behind this integration is to compensate for the significant spatial compression (down to $1/8$ of the original dimensions) that occurs during feature extraction. By "pasting" the early-stage pixels onto the "smart" multi-scale features, the model recovers precise boundary information. Following concatenation, a 3×3 convolutional layer is applied to smooth the transition between these two distinct feature sets before the final classification head. This ensures that the global context provided by the ASPP logic is grounded by the local structural accuracy of the original input.

3. Dataset

For this project, we utilize the Cityscapes dataset [3], a large-scale benchmark focused on semantic understanding of complex urban street scenes. The data consists of high-resolution video sequences recorded from a vehicle moving through 50 different cities, providing a diverse set of real-world scenarios. We sourced the dataset via Kaggle, which contains 3,475 images, pre-divided into training and validation sets with high-quality, pixel-level annotations. The original training set contained 2,975 images and masks, which we divided into training and validation set due to restrictions on accessing the real CityScapes testing set. We assigned 80% of these images to our training set and the remaining 20% to our validation set. We used the original validation set as our test set. Overall, our data set up was 3,475 images, with 2380, 595 and 500 images assigned to the training, validation and testing set, respectively.

Cityscapes is particularly well-suited for our architecture because it presents a significant challenge in terms of scale variation; the model must simultaneously recognize small, distant pedestrians and large, proximal structures like buildings and roads [3]. This environment stresses the need for both a large receptive field and fine spatial detail, making it the ideal benchmark for the DenseASPP architecture, which was originally designed and validated using this specific dataset [5].

3.1. Data Preprocessing and Augmentation

The raw Cityscapes data required several preprocessing steps to ensure compatibility with our model and hardware constraints. Although the dataset defines 30 distinct classes, we follow the standard evaluation protocol by mapping these into 19 representative categories (e.g., road, sidewalk, car, person) and grouping irrelevant or rare classes into a "void" label (ID 255) using a custom mapping function [3].

Furthermore, because the original images are 2048×1024 pixels, which is computationally prohibitive for deep training, we implemented a pipeline that instead takes random crops of images of 512×512 resolution. To improve the model's ability to generalize and prevent overfitting, we also applied real-time data augmentation during training, including random horizontal flipping and random brightness, contrast and saturation jittering.

4. Method

We mainly worked with three architectures. The first is a reproduction of the DenseASPP model developed by Yang [5]. The second and third are variants of this architecture, where we implemented the CBAM tool and a system of encoder-decoder. This approach has the intention of slowly increasing the complexity of the model to mea-

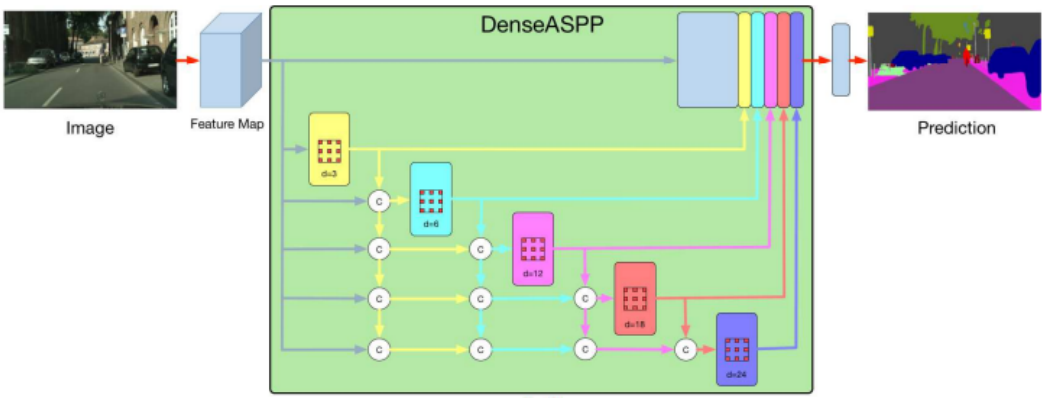

Figure 1. DenseASPP Cascade architecture adapted from Yang [5]

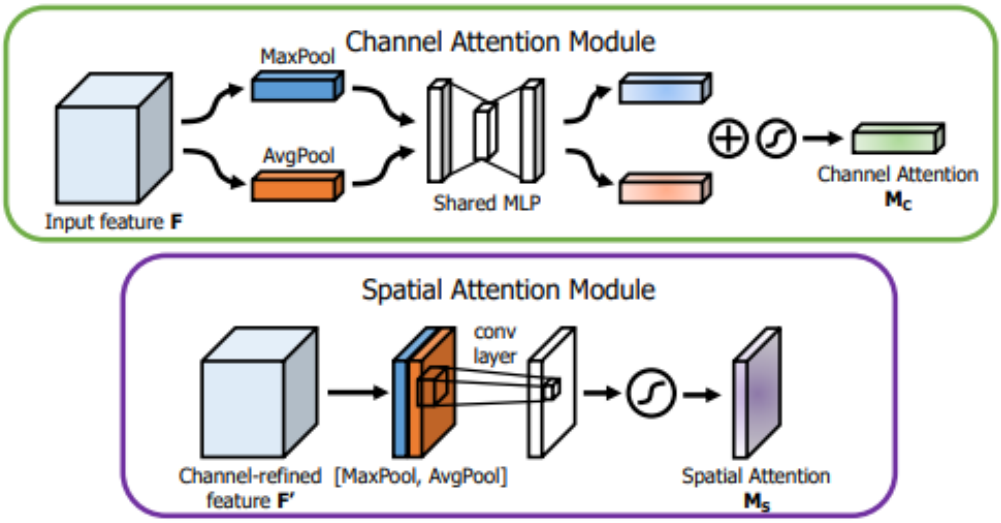

Figure 2. CBAM architecture adapted from Woo [4]

sure how much improvement can be obtained. This implementation is inspired on the results obtained by Woo [4], where the use of the CBAM improved the performance of CNN architectures, such as RESNET50, as well as the simplicity of its structure. Additionally, the use of encoder-decoder has demonstrated promising results when applied to ASPP, according to Chen [2]. In this work, we composed the two methods to exploit their respective advantages. Details about each architecture will be explained further in the following subsections.

4.1. Architecture 1: DenseASPP or "Vanilla" model

The architecture is composed by three elements: pre-trained DenseNet121, the DenseASPP Block and the Head-Block. From the pre-trained DenseNet121 we simply extracted the output from an early layer, specifically when the resolution is at 1/8 of the input's size and at 512 channels, which occurs on the layer "pool3 relu". This part of the architecture will be referred as Backbone, and all its weights will remain untrainable or frozen.

The DenseASPP block is formed by five atrous convolutional layers with the following dilation rates: 3, 6, 12, 18 and 24. They are set up as a cascade structure where the output of each of these layers is concatenated with the input feature map and all the outputs from earlier layers. The final output is a feature map produced by the concatenation of the initial feature map and the output of all layers. Because of this we obtain a feature map with multi-rate and multi-scale atrous convolutions. The visual representation of this block is in Figure 1.

The head block performs 'classification' by performing a final 1×1 convolution with 19 filters, which would output 19 logit channels which are interpreted as likelihoods of belonging to a certain class.

4.2. Architecture 2: DenseASPP + CBAM

In this architecture we aim to improve our Vanilla version by including a CBAM. This module is divided in two submodules, one focus on channel attention and the second on spatial attention.

The Channel Attention Module integrates outputs from max-pooling and average pooling in a shared network with a sigmoid activation function. This network is composed of a multi-layer perceptron (MLP) with one hidden layer. This channel focuses on the simultaneous implementation of both pooling techniques.

On the other hand, the Spatial Attention Module applies average-pooling and max-pooling along the channel axis and concatenates them to generate an efficient feature descriptor. In this module, the pooling techniques are applied sequentially.

Finally, both modules are implemented together. So, the output of the Channel Attention Module corresponds to the input of the Spatial Attention Module. It's also relevant to mention that this CBAM is integrated on the architecture just after the DenseASPP Block, and before the Head Block. The intention is to refine our features just before the classification task. A visual representation of this module is found in Figure 2.

4.3. Architecture 3: DenseASPP + CBAM + encoder-decoder

Our last and more complex architecture, aims to combine the best advantages of both promising techniques: CBAM and encoder-decoder. This architecture takes the Architecture 2 and includes an encoder-decoder during the construction of the model.

First, the encoder takes low level and high level features from the Backbone. A low level feature is still high in resolution since is still on the early layers of the model, while the high-level feature contains low-resolution images due to all the downsampling suffered during the convolutions. Secondly, the high level features are passed into the DenseASPP block and later through the CBAM block to refine its features as in the Architecture 2. Finally, the refined version of our low and high level features is concatenated together and smoothed by applying some last convolutional layer. The output of this system is provided to the Head Block to perform the usual logic classification.

4.4. Training details

In the vanilla model we have a total of 2,905,344 parameters, from which 1,581,888 are trainable. In our second architecture, the number of parameters increased to 3,079,435 with 1,755,979 classified as trainable. This is a normal change caused by the addition of the CBAM. However, for the third architecture there was a considerable jump to 13,535,182 parameters, from which 13,439,502 are trainable.

Additionally, strong data augmentation techniques were necessary to avoid overfitting and force our model to improve in generalization. These techniques included the use of uniform random jittering in brightness in a range of [-10,+10], applied to all pixels. The contrast tool scales the pixel level by generating a random number between [0.5, 1.5]. When this number is <1, it flattens contrast, but if it is >1, it exaggerates the differences. The last technique was random saturation on the color-channel level. Applies similar intervals than contrast, but in this case, numbers <1 make collors duller, and >1 more vivid.

Another relevant characteristic of our training was the use of a polynomial decay function to reduce the learning rate, as implemented by Yang [5]. The function is defined as: $(1 - \frac{epochs}{maxEpochs})^9$. However, a second function was implemented during the experimentation phase, where $1e^{-}5$ was defined as minimum value for the learning rate. So, the function was returning the $max(lr, min_lr)$, in this way we stabilize the learning rate once it has reached a low value, and avoid dead learning rates in later epochs.

The loss was measured using a Sparse Categorial Cross Entropy function from tensor flow, specifying the use of logits inside the definition of the loss object. In this work, the accuracy metric is misleading since we are applying per pixel classification, and classes like 'road' may occupy most of the space in the image, leading to high accuracy scores even if the model is classifying everything with the 'road' label. For this reason, the metric chosen was the mean Intersection over Union (mIoU). The metric is defined by the overlap between the predicted segmentation and the ground truth, divided by the total area covered by the union of the two. A modified version of mIoU was created, since standard mIoU operates on single-channel label maps, not on logits. Considering that the model outputs 19-channel logits (one per class), we apply argmax across channels to obtain per-pixel class predictions, which are then used to compute mIoU.

Other tools like Adam Optimizer, early stopping and weight decay were used to improve our training. This last with a value of $1e^{-}5$.

5. Experiments

Our first experimental phase was performed for 50 epochs for all the models and using the polynomial decay function to reduce the learning rate. Table 1 shows the mIoU results obtained in this initial phase. In this table, we call effective epochs to the epoch from which the weights are taken, considering that our early stop tool saves the best weights of the training. From these results, it is clear that the CBAM is not performing as expected, since it reduced the mIoU scores when it performed by itself. On the other

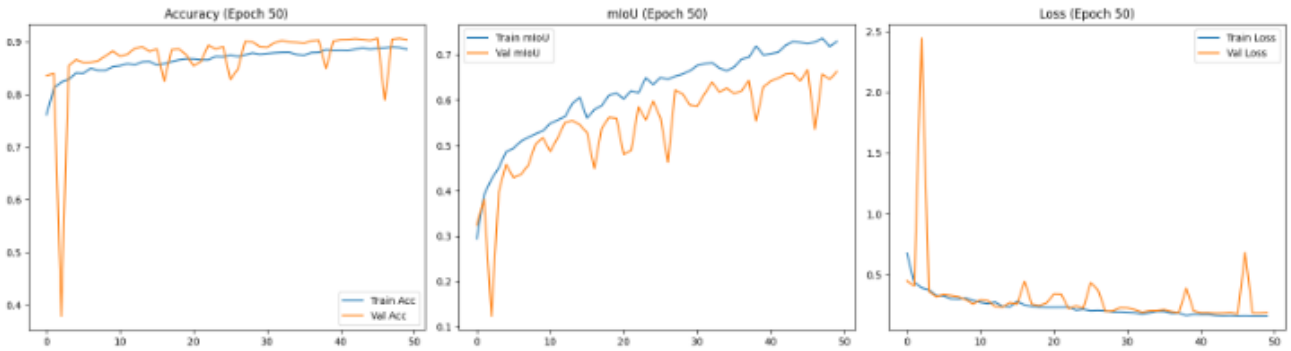

Figure 3. Training process of Architecture 3 for 50 epochs

Model	Test mIoU	Effective Epochs
Architecture 1	0.625	50
Architecture 2	0.508	50
Architecture 3	0.6188	46

Table 1. Comparison of mIoU scores on the three architectures after 50 epochs training

Model	Test mIoU	Effective Epochs
Architecture 1	0.56	56
Architecture 3	0.601	60

Table 2. Comparison of mIoU scores on the three architectures after 60 epochs training and unfreezing 30 layers of Backbone

.

hand, the encoder-decoder system provided similar results than our Vanilla model. In Figure 3 it can be observed the accuracy, mIoU and loss function of the Architecture 3 for over 50 epochs. The peaks in these plots show certain instability in the results; however, the loss function was decreasing and the mIoU increasing, which indicates that the model is still learning.

Based on the results obtained in the initial phase, different strategies were followed, such as the implementation of the improved polynomial decay function, which prevents extreme decay of the learning rate. Additionally, 30 layers of the Backbone algorithm were unfrozen after 20 epochs of training, with the purpose of refining its weights. So, in this second phase models were trained 20 epochs as in the previous phase, with the only change of the improved polynomial decay, and for the last 40 epochs the models kept a constant learning rate and training included the weights of the last layers of Backbone. However, since Architecture 2 did not provided meaningful results, we included only the vanilla model and architecture 3 with the encoder-decoder in this phase. These results can be observed in Table 2.

The previous results actually showed a small decrease in the mIoU values, which can be explained by the unfreezing of the Backbone layer. A key point to consider about this approach is that by unlocking these layers, we are actually increasing the number of parameters to learn. Since more features to learn can be translated into longer training, we consider the provided outputs are consistent with the methodology implemented.

6. Conclusion

In this work, we analyzed the effect of different elements known for their effectiveness when applied in CNN models, such as CBAM and an encoder-decoder. In this process we faced the following challenges: insignificant learning rate for long trainings, restricted dataset and computational costs.

Firstly, we identified that the original polynomial decay function was inefficient for long trainings, such as 100 epochs, due to a vanishing learning rate. Since this function slowly reduces this parameter per epoch, the last epochs were seriously affected and learning was stuck. Secondly, since the testing set of the cityscapes is restricted, we couldn't access to those 1,525 images. These elements represent a big proportion of the full set, and limited our results compared to those by state-of-art. Thirdly, the running times of each model ranged between 4-5 hours with the use of GPU accelerators. This limitation in resources minimized the experimental part of this project.

However, we found potential for an encoder-decoder system for refining Backbone's weights. Since this system takes as input features from low and high level in this Architecture, by unfreezing some layers we enhance the performance of this element.

Opportunities for further improvement that we identified include: Performing data augmentation techniques that may 'increase' our number of data or allow for proper reuse, optimizing and experimenting with different transfer learning techniques from the backbone, or using additional relevant datasets like ImageNet to have more data to train and test on.

References

[1] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017.

[2] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European conference on computer vision (ECCV)*, pages 801–818, 2018.

[3] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3213–3223, 2016.

[4] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. Cbam: Convolutional block attention module. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 3–19, 2018.

[5] Maoke Yang, Kun Yu, Chi Zhang, Zhiwei Li, and Kuiyuan Yang. Denseaspp for semantic segmentation in street scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.